



# EXPLOITS UNIVERSITY

BSc INFORMATION COMMUNICATION AND TECHNOLOGY

**BICT 3202 · OBJECT-ORIENTED ANALYSIS AND DESIGN**

## WEEK 16

**Comprehensive Revision, Integrated Case Study and Examination Preparation**

**PROGRAMME**

BSc Information Communication and Technology

**COURSE CODE / YEAR**

BICT 3202 · Year 3

**LECTURER**

Francis Fweta — ICT Lecturer

**DELIVERY**

2h Lecture · 1h Tutorial · 1h Practical · 10h Independent Learning

*Prepared by Francis Fweta · Exploits University · Week 16 of 16*

## Contents

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| 1. Introduction to Week 16                                              | 4  |
| 2. Week 16 Learning Outcomes                                            | 4  |
| 3. The Entire Module in One Picture                                     | 5  |
| PART A — WHAT IS OOAD?                                                  | 5  |
| 4. Object-Oriented Analysis and Design                                  | 5  |
| PART B — OBJECT-ORIENTED PRINCIPLES REVISION                            | 6  |
| 5. Object, Class, Encapsulation and Abstraction                         | 6  |
| 6. Inheritance and Polymorphism                                         | 7  |
| PART C — UML REVISION                                                   | 8  |
| 7. What Is UML, and Why Do We Need It?                                  | 8  |
| 8. UML Diagram Families                                                 | 8  |
| PART D — USE-CASE REVISION                                              | 8  |
| 9. Actor, Use Case, Include and Extend                                  | 8  |
| PART E — CLASS DIAGRAM REVISION                                         | 9  |
| 10. Class, Attribute, Operation and Visibility                          | 9  |
| PART F — RELATIONSHIPS REVISION                                         | 10 |
| 11. Association, Multiplicity, Aggregation, Composition, Generalisation | 10 |
| PART G — DYNAMIC MODELLING REVISION                                     | 11 |
| 12. Event, State, Transition and the State Diagram                      | 11 |
| PART H — BEHAVIOURAL MODELLING REVISION                                 | 11 |
| 13. Activity, Sequence and Communication Diagrams                       | 11 |
| PART I — FULL MODEL INTEGRATION                                         | 12 |
| 14. The Most Important Week 16 Concept                                  | 12 |
| PART J — UML MODEL CONSISTENCY                                          | 13 |
| 15. What Does Consistency Mean?                                         | 13 |
| PART K — UNIFIED SOFTWARE DEVELOPMENT PROCESS REVISION                  | 14 |
| 16. USDP/UP and Its Four Phases                                         | 14 |
| PART L — ITERATIVE DEVELOPMENT                                          | 14 |
| 17. What Is an Iteration?                                               | 14 |



---

|                                              |    |
|----------------------------------------------|----|
| PART M — REUSE REVISION                      | 15 |
| 18. What Is Software Reuse?                  | 15 |
| PART N — CASE STUDY FOR FINAL REVISION       | 15 |
| 19. Scenario: Mobile Money System            | 15 |
| PART O — PRACTICAL REVISION                  | 16 |
| 20. Practical Exercise and Challenge         | 16 |
| PART P — EXAMINATION REVISION                | 16 |
| 21. Common Theory and Practical Questions    | 16 |
| PART Q — HOW TO APPROACH AN EXAM SCENARIO    | 17 |
| 22. Never Start Drawing Immediately          | 17 |
| PART R — COMMON STUDENT MISTAKES             | 17 |
| PART S — MASTER REVISION TABLE               | 18 |
| 23. All Major Concepts                       | 18 |
| PART T — THE 16-WEEK JOURNEY                 | 18 |
| 24. What Students Have Covered               | 18 |
| 25. Tutorial, Practical and Mock Examination | 19 |
| 26. Exam Answering Strategy                  | 19 |
| 27. Final Week 16 Takeaway                   | 20 |
| 28. Final Revision Checklist                 | 20 |
| 29. References and Further Reading           | 20 |
| 30. Conclusion of the 16-Week Module         | 21 |

## 1. Introduction to Week 16

Welcome to the final teaching week of Object-Oriented Analysis and Design. This week is deliberately different: we are not introducing another isolated topic. Week 16 answers the most important question: *can the student take everything learned throughout the module and solve a new object-oriented analysis and design problem independently?*

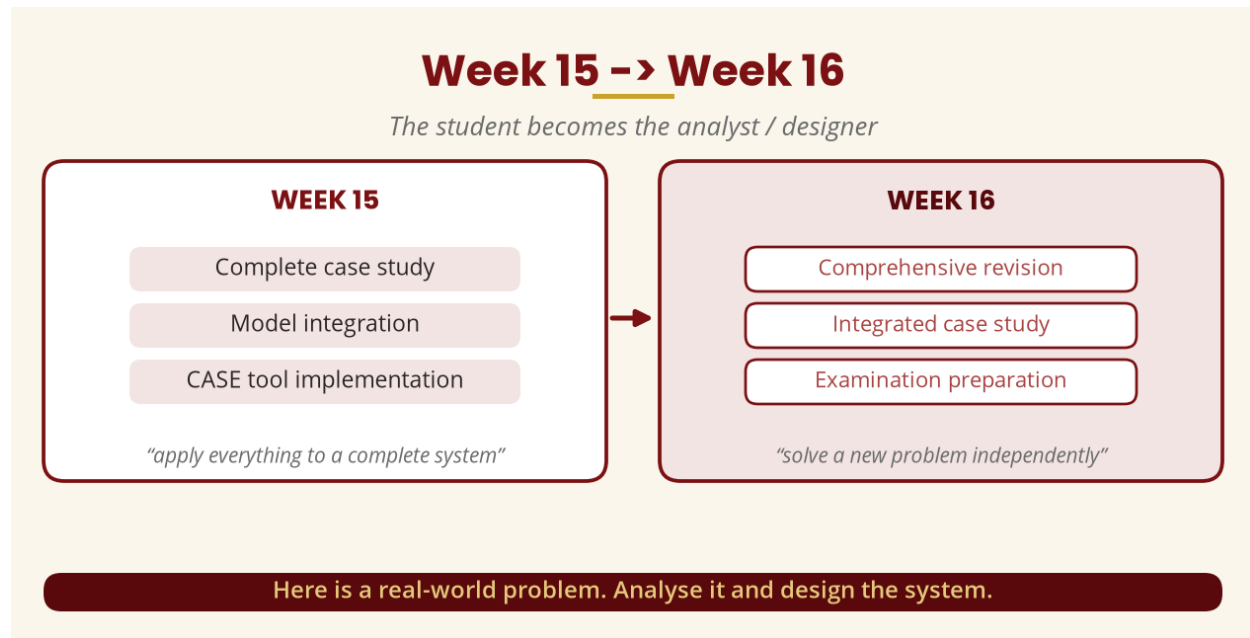


Figure 1 — Week 15 -> Week 16

## 2. Week 16 Learning Outcomes

By the end of this week, a student should be able to:

- Explain the principles of OOAD and the strengths and limitations of object-oriented development.
- Distinguish analysis from design; explain the purpose of UML and its diagram families.
- Identify actors, use cases, objects and classes from a problem statement.
- Construct use-case, class, state, activity, sequence and communication diagrams.
- Explain association, aggregation, composition, generalisation, inheritance and polymorphism.
- Explain the Unified Software Development Process — phases, iterations, workflows, artefacts.
- Combine object, dynamic and behavioural models; evaluate model consistency.
- Apply OOAD to an unfamiliar problem and answer examination questions effectively.

### 3. The Entire Module in One Picture

Students should be able to reproduce this conceptual journey from memory: understand the problem -> business needs -> requirements -> use cases -> identify objects/classes -> object model, dynamic model and behaviour model (classes, states, interactions) -> analysis model -> object-oriented design -> architecture/components -> implementation. **These are connected stages and views, not unrelated examination topics.**

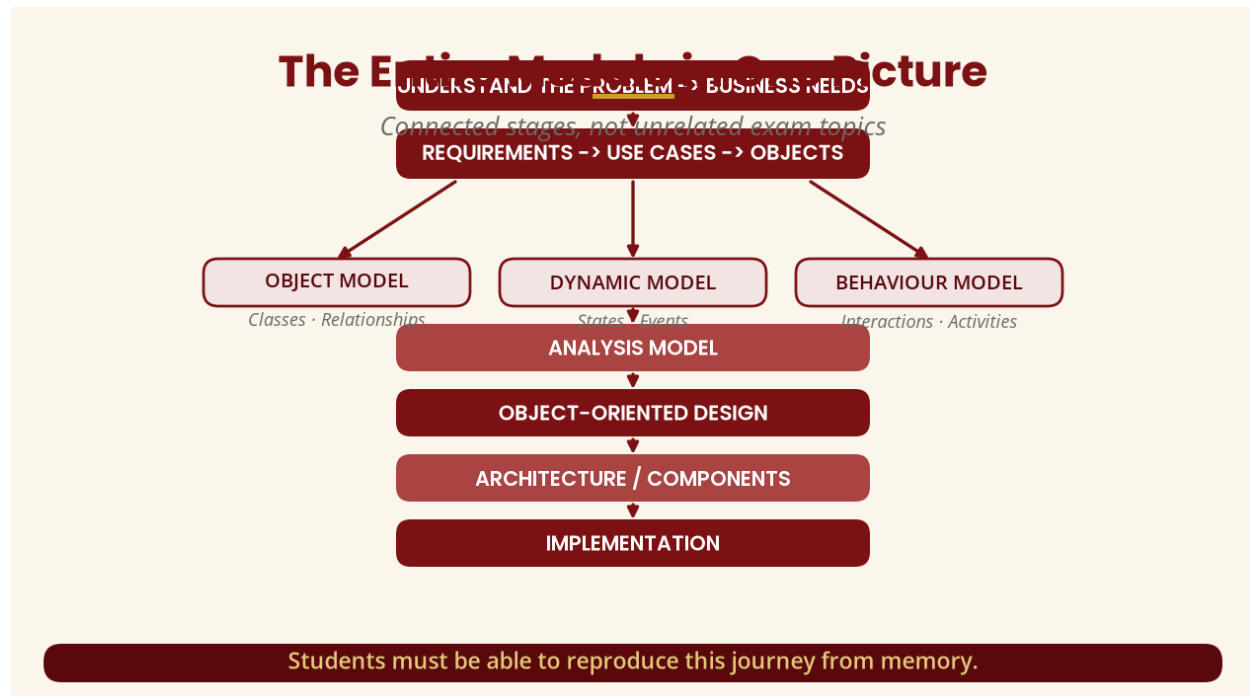


Figure 2 — The entire module in one picture

## PART A — WHAT IS OOAD?

### 4. Object-Oriented Analysis and Design

**Object-oriented analysis** asks: *what does the system need to do, and what objects/classes exist in the problem domain?* For a library system the analyst may identify Student, Book, Librarian, Loan and Library, then investigate what a student does, what a book is, how a book is borrowed and returned, who may borrow, and the relationships between students and loans.

**Object-oriented design** asks: *how should the software be structured so those requirements can actually be implemented?* Where analysis identifies Student, Book and Loan, design may introduce LoanService, BookRepository, StudentRepository, NotificationService and LoanController. Therefore: **analysis focuses on understanding the problem; design focuses on constructing a software solution.**

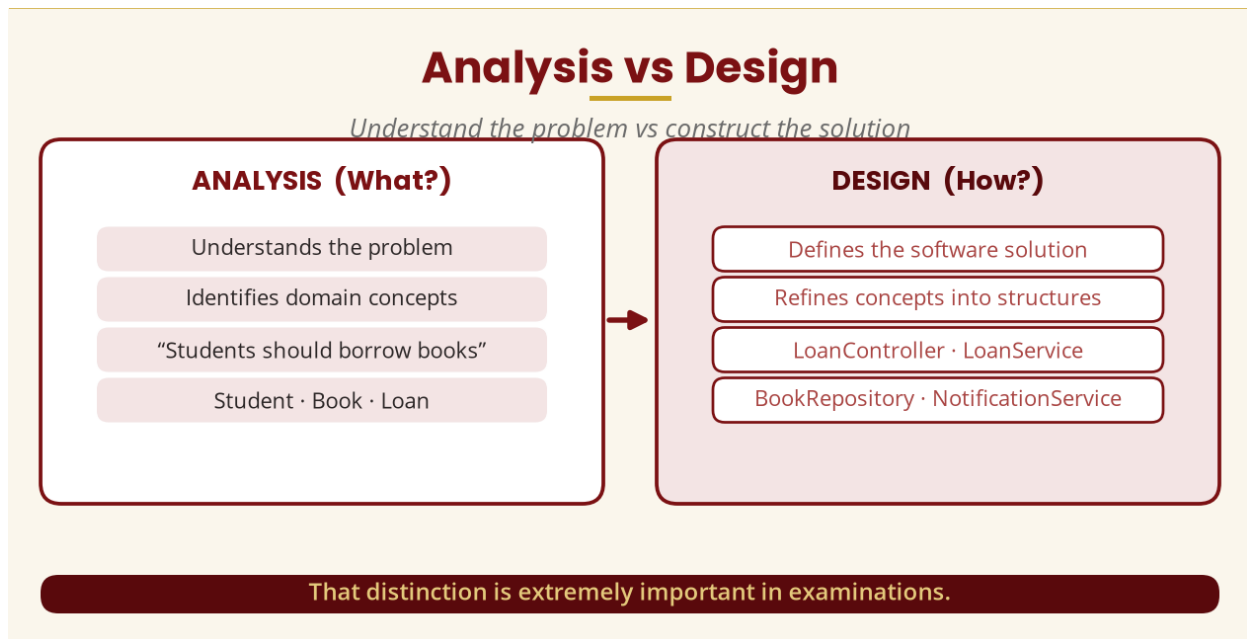


Figure 3 — Analysis vs design

| Analysis                   | Design                                |
|----------------------------|---------------------------------------|
| Understands the problem    | Defines the solution                  |
| What does the system need? | How will the system do it?            |
| Identifies domain concepts | Refines them into software structures |
| Focuses on requirements    | Focuses on implementation structure   |
| "What?"                    | "How?"                                |

## PART B — OBJECT-ORIENTED PRINCIPLES REVISION

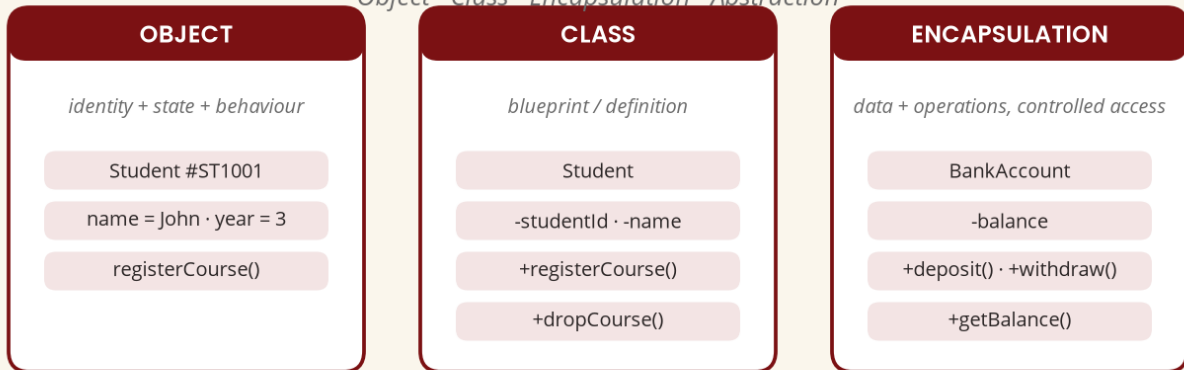
### 5. Object, Class, Encapsulation and Abstraction

An **object** represents a particular entity with **identity, state and behaviour** — Student #ST1001 with name = John, programme = ICT, year = 3 and behaviour registerCourse(), dropCourse(), viewResults(). A **class** is the blueprint from which objects are created; the object is an instance of that blueprint.

**Encapsulation** keeps an object's data and the operations that control that data together while controlling inappropriate direct access — a customer should not set balance = -500000 directly; the BankAccount controls how balance changes through deposit(), withdraw() and getBalance(). **Abstraction** focuses on important characteristics while hiding unnecessary detail — an ATM shows Withdraw, Deposit and Check Balance without exposing database queries, encryption or server communication.

## Core OO Concepts

*Object · Class · Encapsulation · Abstraction*



**Abstraction: focus on important characteristics; hide unnecessary details.**

Figure 4 — Core object-oriented concepts

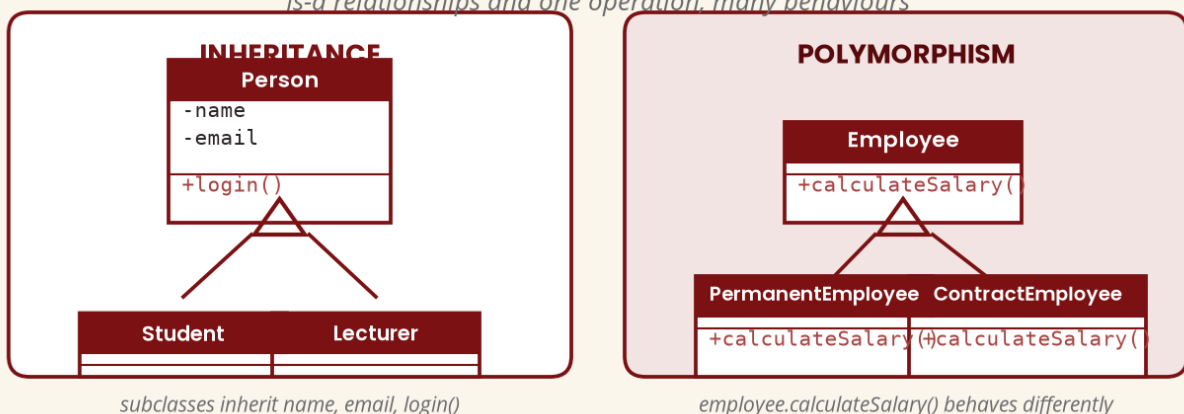
## 6. Inheritance and Polymorphism

**Inheritance** lets a specialised class inherit characteristics from a more general class — Student and Lecturer inherit name, email and login() from Person. **Polymorphism** means the same operation/interface can behave differently depending on the object involved — PermanentEmployee and ContractEmployee each implement calculateSalary() differently, so employee.calculateSalary() behaves differently per object.

**Examination distinction:** generalisation is the UML modelling relationship; inheritance is the object-oriented programming mechanism commonly used to implement it.

## Inheritance & Polymorphism

*Is-a relationships and one operation, many behaviours*



**Generalisation is the UML relationship; inheritance is the implementation mechanism.**

Figure 5 — Inheritance and polymorphism

## PART C — UML REVISION

### 7. What Is UML, and Why Do We Need It?

**UML** is the Unified Modeling Language — a standard modelling language used to represent and communicate software/system models. The Object Management Group (OMG) lists UML 2.5.1 as a formal specification (adopted December 2017). Remember: **UML is a modelling language, not a programming language**. A complicated banking transaction described in a paragraph is hard to visualise; a diagram makes relationships and behaviour much easier to communicate.

### 8. UML Diagram Families

**Structural diagrams** show what the system is made of — class, component, deployment and package diagrams. **Behavioural diagrams** show what the system does — use-case, activity and state-machine diagrams. **Interaction diagrams** focus on communication between participants — sequence and communication/collaboration diagrams.

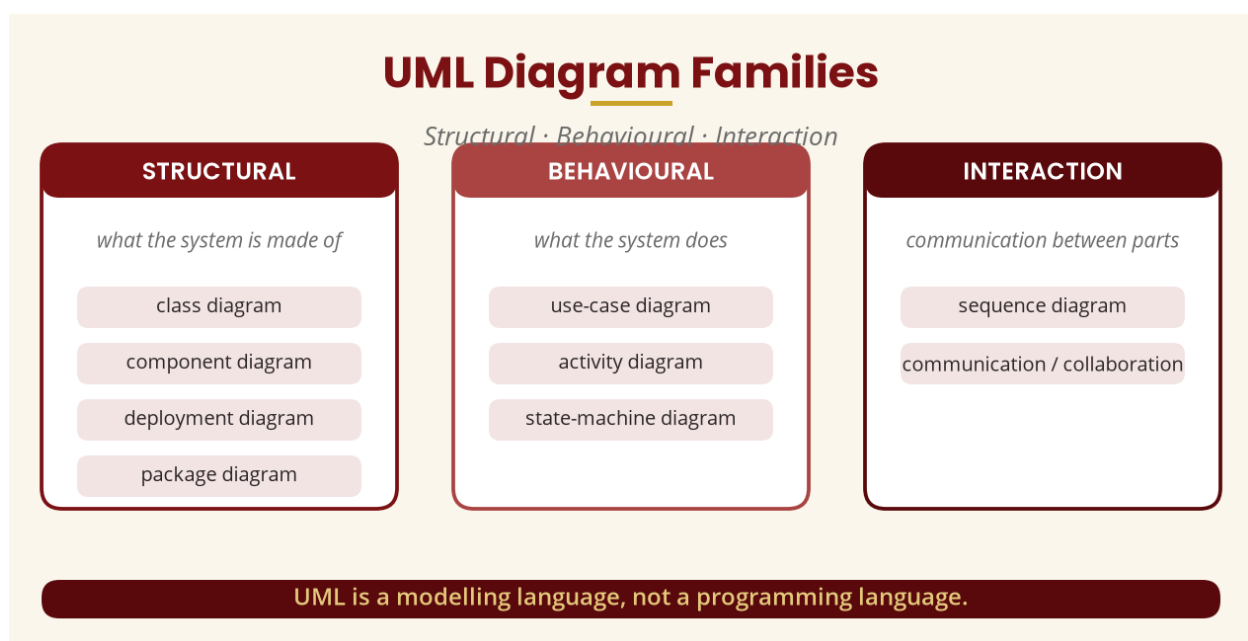


Figure 6 — UML diagram families

## PART D — USE-CASE REVISION

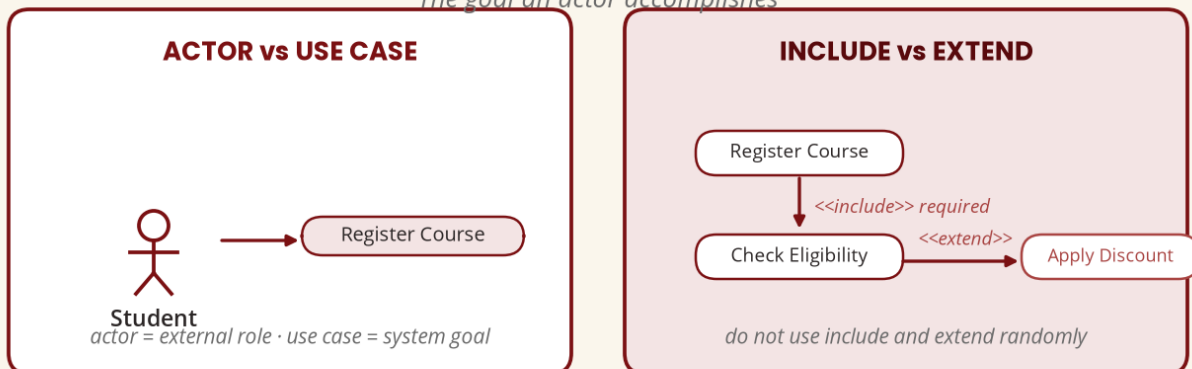
### 9. Actor, Use Case, Include and Extend

A **use case** represents a goal an actor wants to accomplish with the system. The **actor** is the external role (Student); the **use case** is the goal (Register Course). <<include>> is used when one use case always incorporates required behaviour (Register Course includes Check Eligibility); <<extend>> is used for optional, conditional behaviour (Make Payment extended by Apply Discount). **Do not use include and extend randomly.**



## Actors, Use Cases, Include and Extend

*The goal an actor accomplishes*



**Include = required behaviour · Extend = optional additional behaviour.**

Figure 7 — Actors, use cases, include and extend

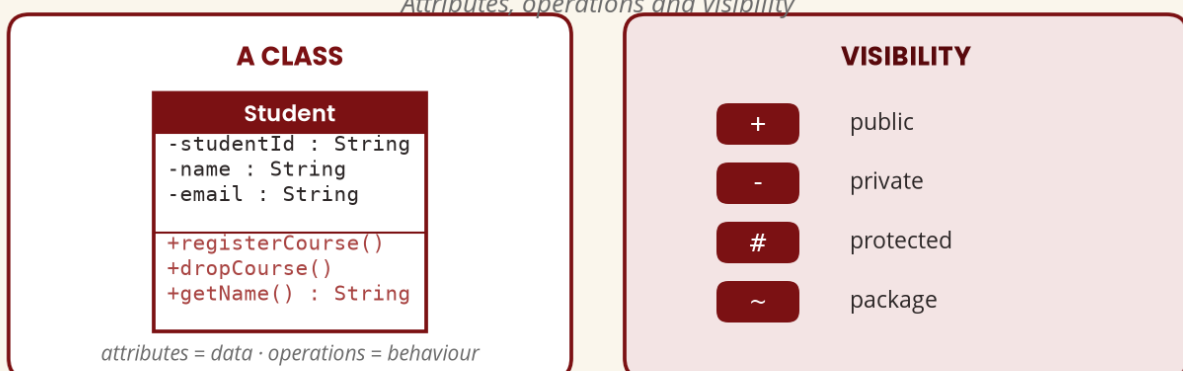
## PART E — CLASS DIAGRAM REVISION

### 10. Class, Attribute, Operation and Visibility

The **class diagram** describes the static structure. An **attribute** is data belonging to a class (studentId, name, email); an **operation** is behaviour or responsibility (registerCourse(), dropCourse()). Common **visibility** symbols: + public, - private, # protected, ~ package — e.g. - studentId : String and + getName() : String.

## Class Diagram Notation

*Attributes, operations and visibility*



**The class diagram describes the static structure of the system.**

Figure 8 — Class diagram notation

## PART F — RELATIONSHIPS REVISION

### 11. Association, Multiplicity, Aggregation, Composition, Generalisation

**Association** is a general relationship between classes (Student — Course). **Multiplicity** says how many objects participate: 1 exactly one, 0..1 zero or one, \* many, 0..\* zero or many, 1..\* one or many — e.g. Student 1 — 0..\* Registration.

**Aggregation** is a weaker whole-part relationship where parts can exist independently (Department o— Lecturer); **composition** is stronger ownership/lifecycle (House \*— Room).

**Generalisation** is an “is-a” relationship (Car and Motorcycle are Vehicles). Choose the diamond from the ownership/lifecycle relationship, not because “the diamond looks nice.”

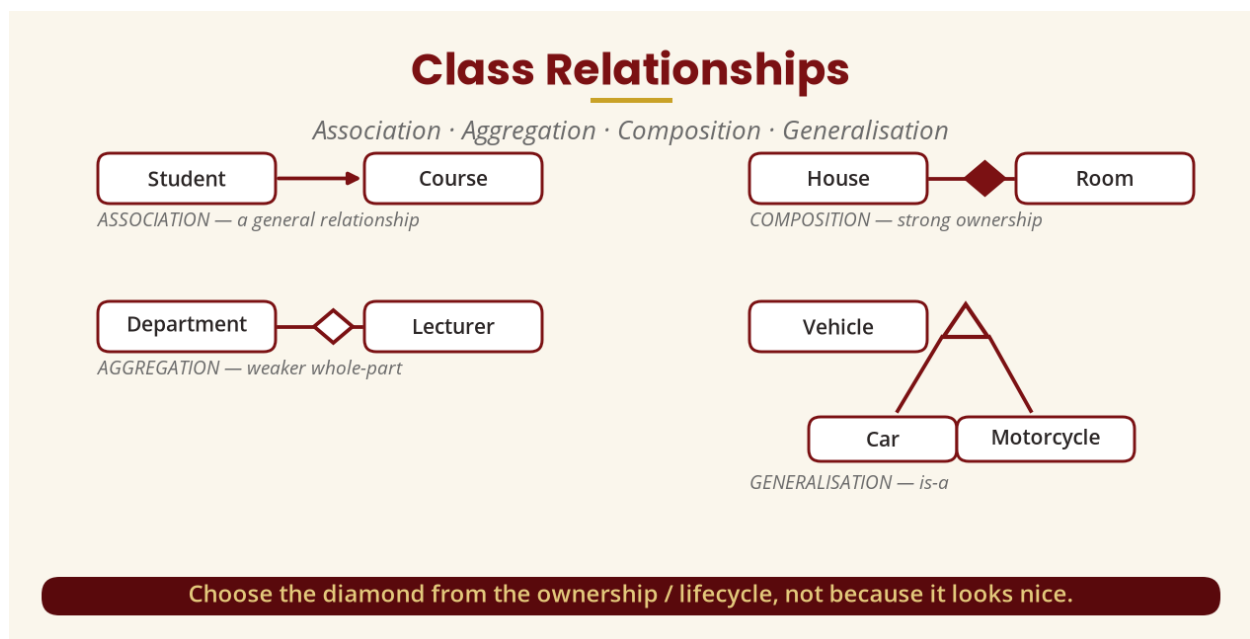


Figure 9 — Class relationships

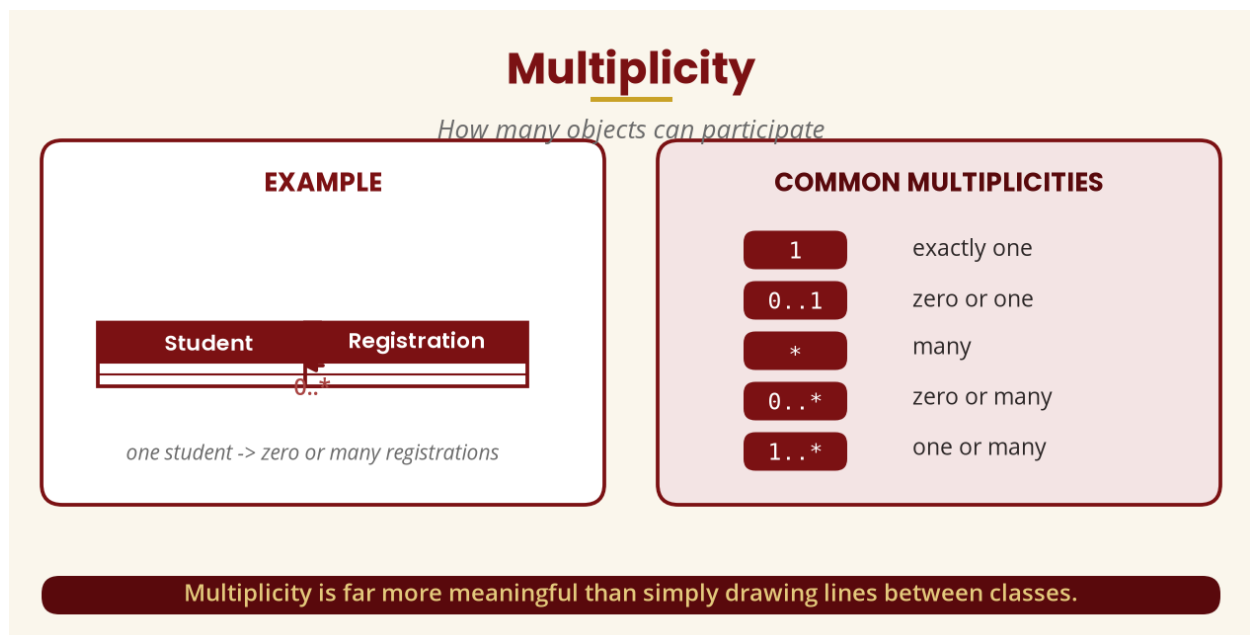


Figure 10 — Multiplicity

## PART G — DYNAMIC MODELLING REVISION

### 12. Event, State, Transition and the State Diagram

Dynamic modelling describes how the system changes over time — “*what happens?*” rather than “*what exists?*”. An **event** causes a change (Course Registration Opens); a **state** is a condition at a point (Open, Closed, Full); a **transition** is movement between states (Available -> Full when the final seat is taken). The Course lifecycle: Created -> Available -> Full -> Closed.

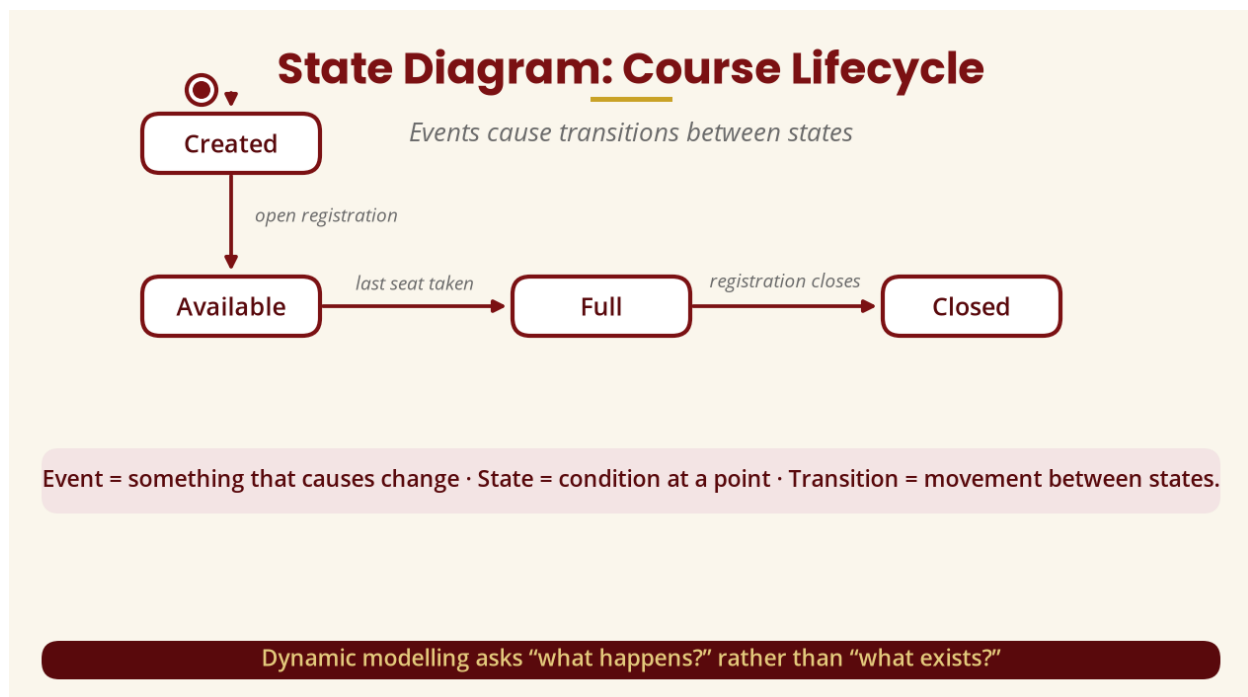


Figure 11 — State diagram: Course lifecycle

## PART H — BEHAVIOURAL MODELLING REVISION

### 13. Activity, Sequence and Communication Diagrams

An **activity diagram** models a workflow (Start -> Login -> Select Course -> Check Eligibility -> Check Capacity -> Register -> End). A **sequence diagram** focuses on *who communicates with whom and in what order* (Student -> RegistrationController -> RegistrationService -> StudentRecord -> Registration). A **communication/collaboration diagram** also models interactions but emphasises objects, relationships and message numbering (1: register(), 2: validate(), 3: save()).

## Sequence Diagram vs Activity Diagram

*A classic examination comparison*

| Sequence                       | Activity                       |
|--------------------------------|--------------------------------|
| Focuses on interactions        | Focuses on workflow            |
| Shows participants / lifelines | Shows activities and decisions |
| Shows message ordering         | Shows process flow             |

Sequence = who talks to whom? · Activity = what happens next?

Figure 12 — Sequence diagram vs activity diagram

| Sequence Diagram             | Activity Diagram         |
|------------------------------|--------------------------|
| Focuses on interactions      | Focuses on workflow      |
| Shows participants/lifelines | Shows activities/actions |
| Shows message ordering       | Shows process flow       |

## PART I — FULL MODEL INTEGRATION

### 14. The Most Important Week 16 Concept

Follow one requirement through the chain: **Requirement** ("students must be able to register for courses") -> **Use Case** (Register Course) -> **Classes** (Student, Course, Registration, RegistrationService) -> **Sequence** (Student -> RegistrationService -> Course -> Registration) -> **State** (Course: Available -> Full) -> **Activity** (Select Course -> Check Eligibility -> Check Capacity -> Register) -> **Design** (Presentation -> Application -> Domain -> Persistence). That is integrated OOAD.

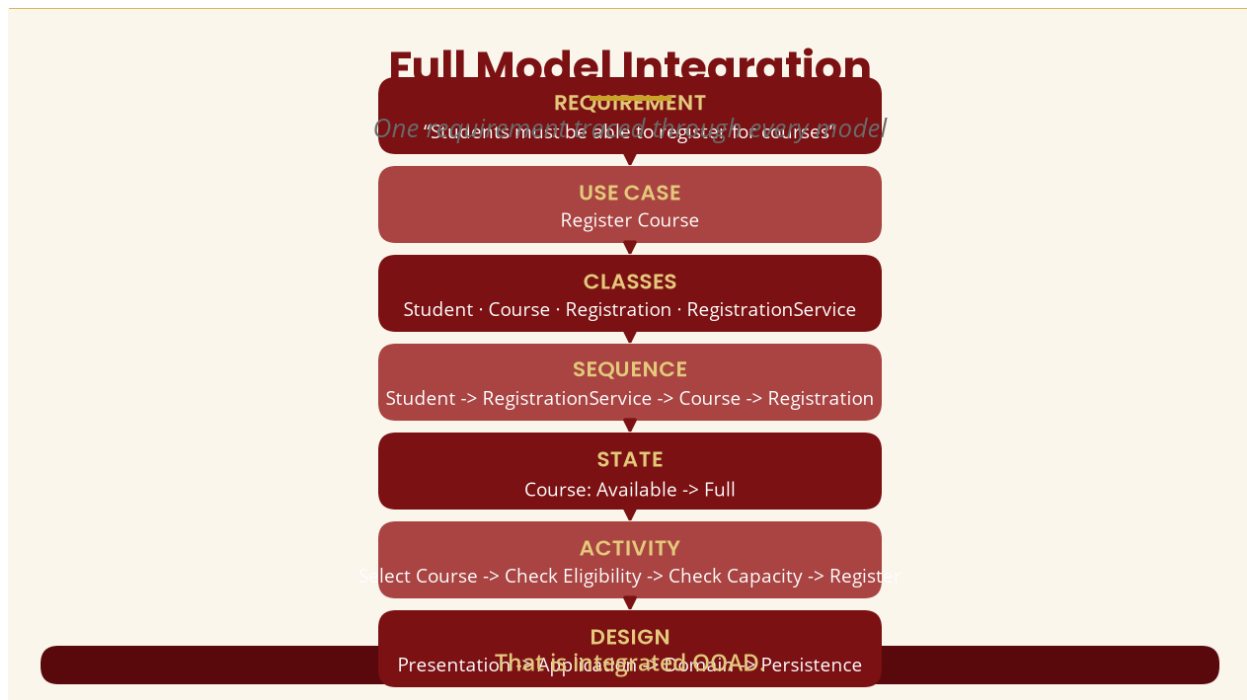


Figure 13 — Full model integration

## PART J — UML MODEL CONSISTENCY

### 15. What Does Consistency Mean?

If the class diagram says `Course.checkCapacity()` but the sequence diagram says `Course.checkAvailableSeats()`, ask: *are these the same operation?* If not, the model may be inconsistent. Perform the six checks: (1) use cases <- requirements; (2) use cases <- classes; (3) sequence <- class; (4) activity <- sequence; (5) state <- behaviour; (6) design <- requirements.

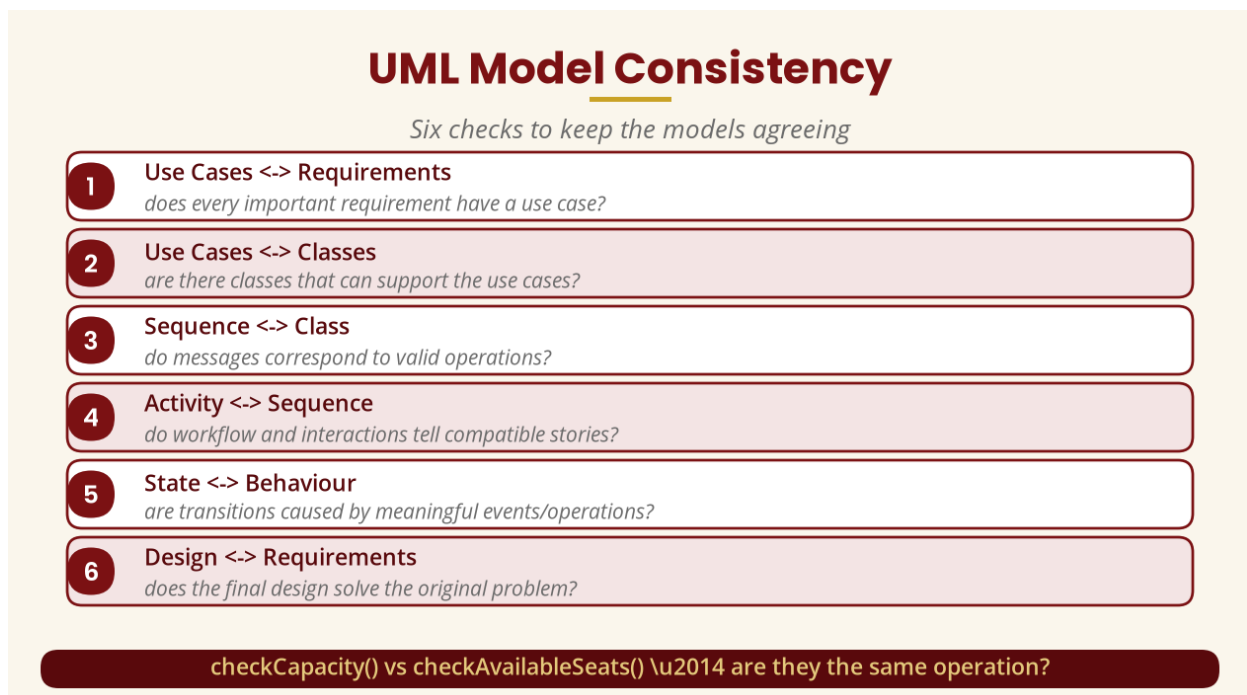


Figure 14 — Six model-consistency checks

## PART K — UNIFIED SOFTWARE DEVELOPMENT PROCESS REVISION

### 16. USDP/UP and Its Four Phases

The Unified Software Development Process (USDP), associated with the Unified Process, is an **iterative and incremental** approach: development is divided into phases and iterations rather than one uninterrupted sequence. IBM's Rational Unified Process documentation describes four phases:

**Inception** — “should we build this system?” (business problem, scope, stakeholders, feasibility, business case). **Elaboration** — “do we understand the problem and architecture?” (requirements, architecture, risk, key use cases, initial design). **Construction** — “can we build it?” (implementation, integration, testing, refinement). **Transition** — “can users use it?” (deployment, acceptance, training, fixes, operational use).

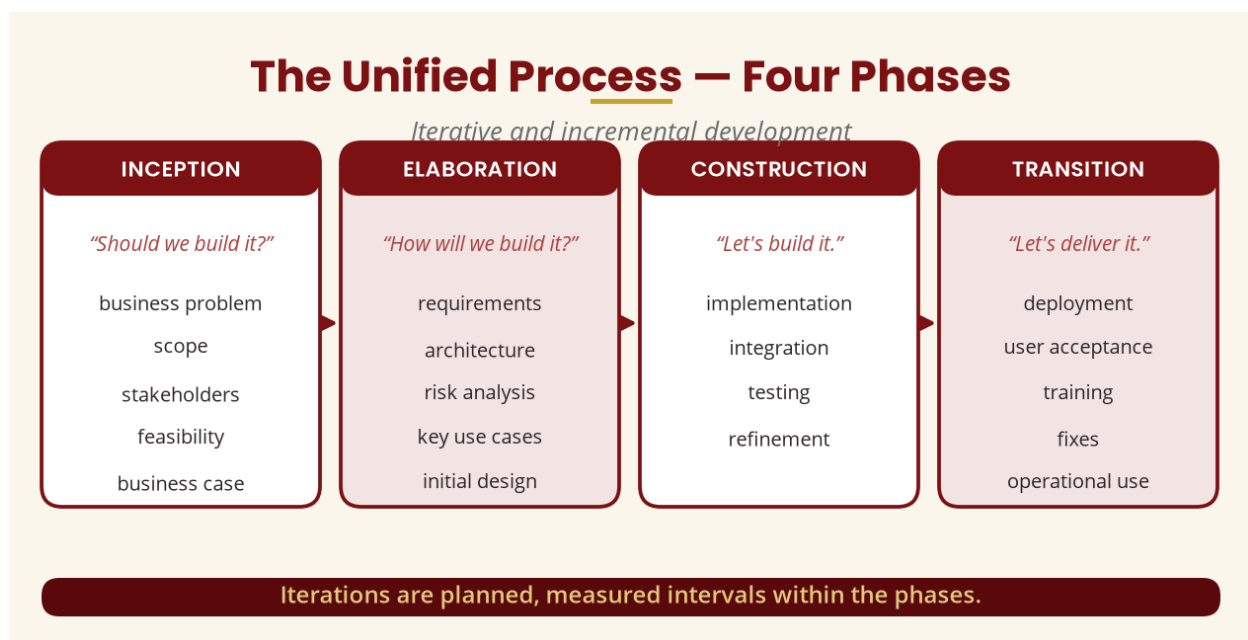


Figure 15 — The four phases of the Unified Process

## PART L — ITERATIVE DEVELOPMENT

### 17. What Is an Iteration?

An **iteration** is a development cycle during which a team produces and evaluates part of the system. Instead of “plan everything -> build everything -> test everything”, an iterative approach cycles through: Iteration 1 (requirements + basic design) -> feedback -> Iteration 2 (more functionality) -> feedback -> Iteration 3 (more functionality) -> feedback -> Iteration 4 (refinement + testing). Iterations are planned, measured intervals within phases that deliver incremental value to stakeholders.

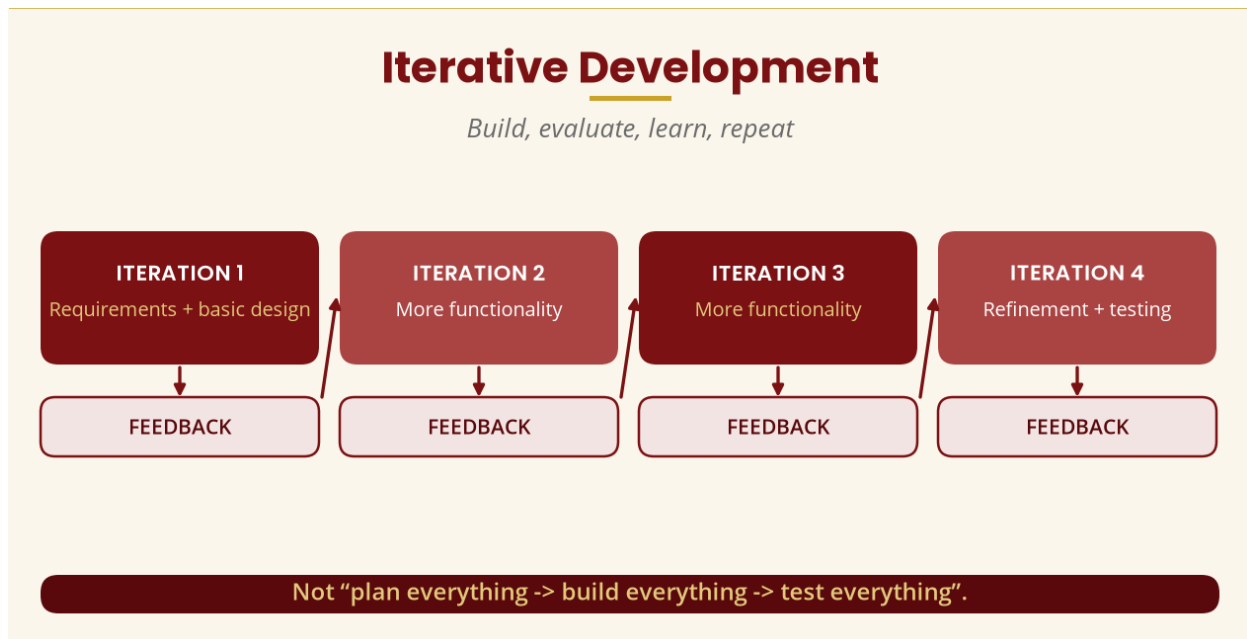


Figure 16 — Iterative development

## PART M — REUSE REVISION

### 18. What Is Software Reuse?

**Software reuse** means using an existing component, class, library, framework, service, design or architecture rather than developing everything from scratch — an authentication library instead of a bespoke mechanism, or a UI framework instead of building every control. The goal: reuse -> less duplication -> less effort -> potentially improved consistency -> easier maintenance. However, reuse should be based on suitability, security, maintainability, compatibility and licensing — not simply because something already exists.

## PART N — CASE STUDY FOR FINAL REVISION

### 19. Scenario: Mobile Money System

A telecommunications company wants a mobile money system. **Users** create accounts, deposit, withdraw, send, receive, check balance and view history. **Agents** deposit and withdraw for customers and view their transactions. **Administrators** manage users and agents, monitor transactions and generate reports.

**Actors:** Customer, Agent, Administrator, Banking System, Notification Service. **Candidate classes:** User, Customer, Agent, Administrator, Wallet, Transaction, Deposit, Withdrawal, Transfer, Notification — plus services AuthenticationService, TransactionService, NotificationService. **Relationships:** User generalises Customer / Agent / Administrator; Customer 1 — 1 Wallet; Wallet 1 — 0..\* Transaction. **Send Money:** sendMoney() -> validateSender() -> checkBalance() -> creditReceiver() -> createTransaction() -> sendNotification(). **Transaction states:** Created -> Pending -> Completed (or Failed).

## Case Study: Mobile Money System

*Actors · inheritance · associations · send-money flow*

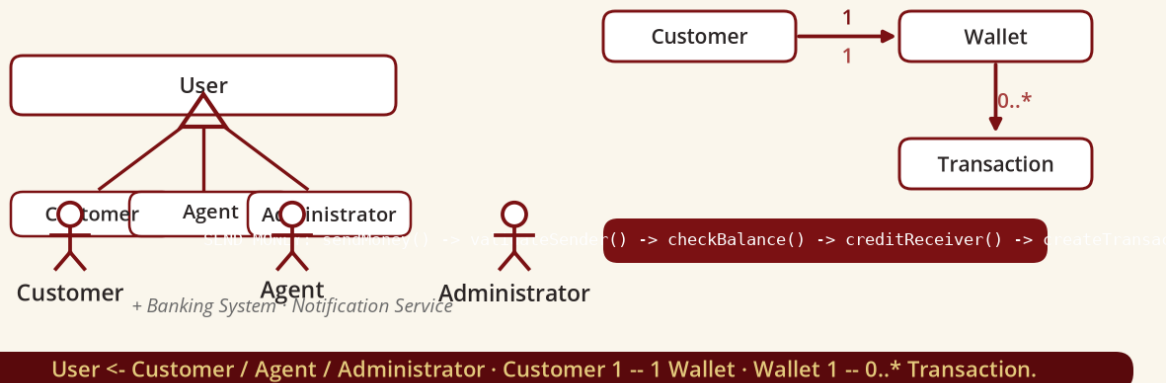


Figure 17 — Mobile money case study

## PART O — PRACTICAL REVISION

### 20. Practical Exercise and Challenge

Use a UML/CASE tool to model the mobile money system with six diagrams: use-case, class, sequence (Send Money), activity (Send Money), state-machine (Transaction) and component. Then deliberately introduce the error: the sequence diagram calls `validateTransaction()` but the class diagram contains `checkTransaction()` — and correct it. **UML modelling is complete when the model is logically consistent, not when the diagram looks beautiful.**

## PART P — EXAMINATION REVISION

### 21. Common Theory and Practical Questions

**Theory:** define OOAD; explain five characteristics of OO systems; distinguish an object from a class; explain encapsulation, abstraction, inheritance and polymorphism; what is UML; structural vs behavioural modelling; association vs aggregation vs composition; generalisation vs inheritance; purpose of a use-case diagram; sequence vs activity diagrams.

**Practical:** given a scenario — identify actors and use cases, draw the use-case diagram, identify classes, draw the class diagram with multiplicities and inheritance, create sequence, activity, state and component diagrams, and explain how the diagrams relate.



## PART Q — HOW TO APPROACH AN EXAM SCENARIO

### 22. Never Start Drawing Immediately

Follow a disciplined process: (1) read the problem twice; (2) underline the nouns; (3) identify possible objects/classes; (4) identify actors; (5) identify actions/goals; (6) convert goals into use cases; (7) build the use-case diagram; (8) develop the class model; (9) choose an important scenario; (10) develop the sequence diagram; (11) develop activity/state models; (12) check consistency.

**The noun technique:** in “a patient books an appointment with a doctor; the receptionist manages patient records and doctors access their schedules”, candidate nouns are Patient, Appointment, Doctor, Receptionist, Patient Record and Schedule; candidate verbs are book, manage and access. This is a useful candidate-identification technique — not a mechanical rule that every noun must become a class.

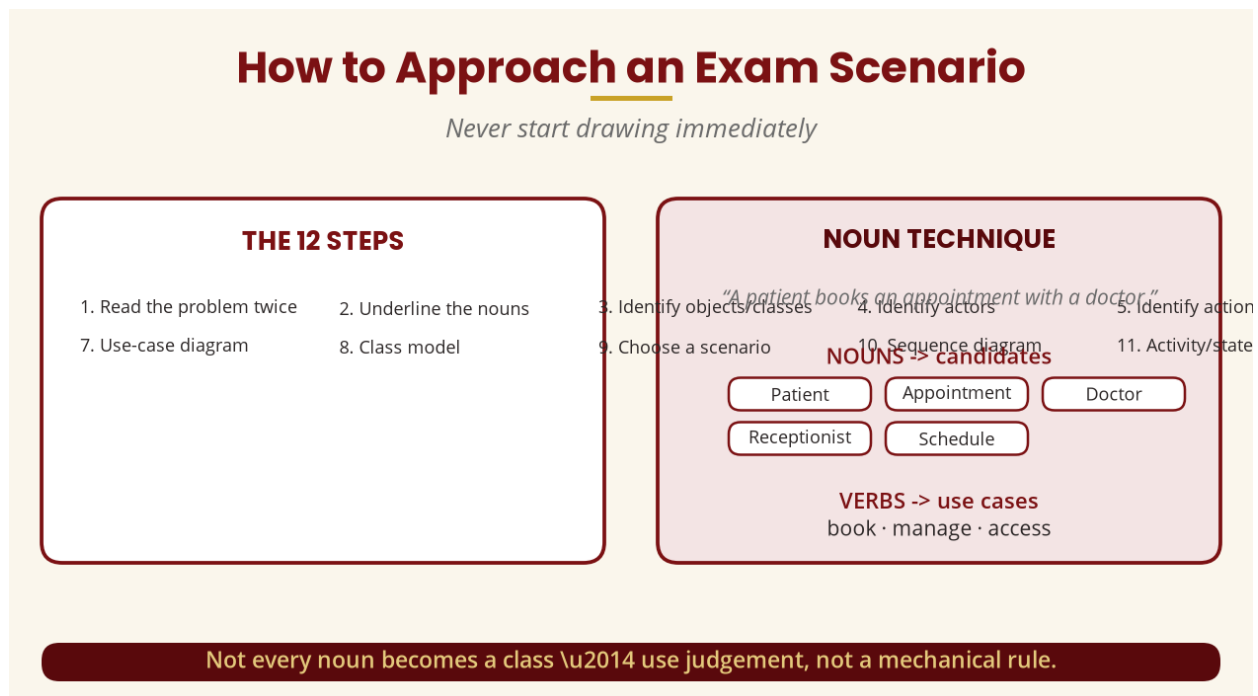


Figure 18 — How to approach an exam scenario

## PART R — COMMON STUDENT MISTAKES

- **Confusing class and object** — Student can be a class; student123 is an object/instance.
- **Every noun becomes a class** — “quickly” is not a class; use judgement.
- **Every verb becomes a method** — ask which object has the responsibility.
- **Confusing include and extend** — include = required; extend = optional.
- **Confusing aggregation and composition** — decide from ownership/lifecycle.
- **Sequence diagrams without class diagrams** — messages need a home in the class model.
- **Beautiful UML, wrong logic** — examiners mark correct modelling, not decoration.

## PART S — MASTER REVISION TABLE

### 23. All Major Concepts

| Topic                                   | Main Question                                                                     |
|-----------------------------------------|-----------------------------------------------------------------------------------|
| Business needs                          | Why does the organisation need the system?                                        |
| Requirements                            | What does the system need to provide?                                             |
| Object / Class                          | What entity exists? / What defines similar objects?                               |
| Encapsulation / Abstraction             | How are data and behaviour controlled? / What details are hidden?                 |
| Inheritance / Polymorphism              | What can a subclass inherit? / How can one operation behave differently?          |
| UML                                     | How do we communicate the model visually?                                         |
| Use case / Class diagram                | What goal does an actor accomplish? / What structure exists?                      |
| Association / Aggregation / Composition | How are classes related? / What whole-part relationship exists?                   |
| Generalisation                          | What is the superclass/subclass relationship?                                     |
| Dynamic model / State / Event           | How does the system change over time?                                             |
| Activity / Sequence / Communication     | What is the workflow? Who communicates, and when?                                 |
| USDP / Reuse / Design / Component       | How is development organised? What can be reused? How is the solution structured? |
| CASE tool / Integration                 | How is modelling supported by software? Do all models tell the same story?        |

## PART T — THE 16-WEEK JOURNEY

### 24. What Students Have Covered

- **Week 1–3:** business needs, business software needs, object technology for business.
- **Week 4–5:** principles of object modelling; UML, its building blocks and mechanisms.
- **Week 6–7:** object modelling I & II — classes, associations, aggregation, inheritance, polymorphism.
- **Week 8:** dynamic modelling — events, states, operations, concurrency.

- **Week 9–10:** behavioural modelling — use cases, sequence, collaboration and activity diagrams.
- **Week 11–12:** the Unified Software Development Process, artefacts and reuse.
- **Week 13–14:** object-oriented design and combining the models.
- **Week 15:** complete OOAD case study and CASE-tool implementation.
- **Week 16:** comprehensive revision, integrated application and examination preparation.

## 25. Tutorial, Practical and Mock Examination

**Tutorial:** an Online Examination Management System — identify actors; at least 10 use cases; at least 10 candidate classes; the use-case diagram; the class diagram; a sequence diagram for Start Examination; an activity diagram for Taking an Examination; a state diagram for Examination (Created -> Scheduled -> Open -> In Progress -> Submitted -> Marked -> Released); identify components; explain how the diagrams relate.

**Practical mini-project:** choose a Hospital, Banking, Mobile Money, University Registration, Library, Hotel, E-commerce or Transport system, and submit: problem statement, objectives, functional and non-functional requirements, actors, use cases, use-case, class, sequence, activity, state and component diagrams, architecture, model consistency analysis and conclusion.

**Mock examination:** Section A — 20 short answers (define OO analysis/design, encapsulation, abstraction, polymorphism, UML, actor, use case, multiplicity, state, event, CASE tool...); Section B — differentiate (class vs object, analysis vs design, aggregation vs composition, generalisation vs inheritance, sequence vs activity, include vs extend); Section C — long answers (four OO principles; the role of UML; the Unified Process); Section D — a full case study with actors, use cases, diagrams and model relationships.

## 26. Exam Answering Strategy

For **definition** questions, do not write only “encapsulation is hiding data” — write the full definition, then give an example. For **explain** questions, use Definition -> Explanation -> Example -> Importance. For **compare** questions, use a table (e.g. class vs object: meaning, memory, example, quantity). For **UML** questions, always identify the requirement, actors, use cases, classes, relationships, behaviour — then check consistency. **Do not randomly draw diagrams.**

## 27. Final Week 16 Takeaway

### THE ENTIRE MODULE IN ONE QUESTION

How can we systematically understand a software problem and represent a solution using objects, classes, UML models and an appropriate development process? The answer: **Business Problem -> Requirements -> Actors -> Use Cases -> Objects/Classes -> Class Model -> Dynamic/State Model -> Behavioural/Interaction -> Analysis -> Design -> Architecture/Components -> Model Integration -> Implementation.** OOAD is not simply about drawing UML diagrams — it is about thinking systematically, identifying what matters, understanding interactions and behaviour, and transforming that understanding into a coherent design developers can implement.

## 28. Final Revision Checklist

- **Object orientation:** can I define object, class, encapsulation, abstraction, inheritance, polymorphism?
- **UML:** can I explain UML and distinguish structural, behavioural and interaction diagrams?
- **Use cases:** can I identify actors and use cases, and use include/extend correctly?
- **Class modelling:** can I identify classes, attributes, operations, associations, multiplicities, aggregation, composition and generalisation?
- **Dynamic & behavioural:** can I model events, states, transitions, activity, sequence and communication diagrams?
- **USDP:** can I explain Inception, Elaboration, Construction, Transition, iterations, workflows and artefacts?
- **Design:** can I distinguish analysis from design, combine the models, identify components, explain layered architecture and reuse?
- **Practical:** can I use a CASE tool, model a new system and check consistency?
- **Examination:** can I explain in my own words, give examples, draw diagrams and justify decisions?

## 29. References and Further Reading

### Primary standards and process frameworks

- Object Management Group (OMG). *Unified Modeling Language (UML), Version 2.5.1*. OMG — formal UML specification and normative documents.
- IBM. *Rational Unified Process (RUP)* — four phases (Inception, Elaboration, Construction, Transition) with iterations as planned intervals.

### Prescribed and recommended texts

- Mach, E. (2019). *Object Oriented Analysis & Design Cookbook: Introduction to Practical System Modeling*.

- Reddy, V., & Korra, S. (2018). *Object-Oriented Analysis and Design Using UML*.
- The Art of Service. (2021). *Object Oriented Analysis and Design: A Complete Guide*.
- Wixom, B., & Dennis, A. (2018). *Systems Analysis and Design*.
- Blokdyk, G. (2021). *Object-Oriented Analysis and Design Standard Requirements*.
- Wixom, B., & Dennis, A. (2020). *Systems Analysis and Design: An Object-Oriented Approach with UML*.

## 30. Conclusion of the 16-Week Module

Week 1 taught students to understand the business problem; Weeks 2–4 developed object technology and modelling; Week 5 introduced UML; Weeks 6–7 developed object/class modelling; Week 8 introduced dynamic modelling; Weeks 9–10 developed behavioural and interaction modelling; Weeks 11–12 introduced the Unified Software Development Process; Weeks 13–14 moved from analysis into design and model integration; Week 15 applied everything through a complete case study. Now, in Week 16, **the student becomes the analyst/designer** — capable of being told “here is a real-world problem” and independently producing Requirements -> Actors -> Use Cases -> Classes -> Relationships -> Interactions -> Behaviour -> Architecture -> Components -> an Integrated UML Model. That is the intended culmination of BICT 3202 Object-Oriented Analysis and Design.